Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 5 March 2014 - Part I: Theory

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the right-linear grammar G below (axiom S), over the terminal alphabet $\{ a, b, c \}$:

$$G \begin{cases} S & \rightarrow & a\ A \\ A & \rightarrow & a\ B \mid \varepsilon \\ B & \rightarrow & c\ S \mid b\ A \end{cases}$$

   Answer the following questions:

   (a) By means of the method of the linear language equations, find a regular expression R equivalent to grammar G.

   (b) By means of the Berry-Sethi method, applied to the regular expression R found before, build a deterministic finite automaton A that recognizes the language $L(G)$ of grammar G.

   (c) (Optional) Build a deterministic finite automaton A' that recognizes the language $\neg L(G)$, i.e., the complement language of $L(G)$.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider the two following grammars G_1 (with axiom S):

$$G_1 \begin{cases} S & \rightarrow & a\ S\ b\ S \ \mid\ a\ S_1\ b\ S_1 \\ S_1 & \rightarrow & c\ S_1\ d\ S_1 \ \mid\ \varepsilon \end{cases}$$

and G_2 (with axiom S):

$$G_2 \begin{cases} S & \rightarrow & S\ e\ S\ f \ \mid\ S_1\ e\ S_1\ f \\ S_1 & \rightarrow & S_1\ c\ S_1\ d \ \mid\ \varepsilon \end{cases}$$

Answer the following questions:

(a) Draw the syntax tree of string $a\,c\,d\,b\,c\,d \in L\,(G_1)$

(b) Write a free grammar G_3 of any type (no matter if it is ambiguous) that generates the concatenation language $L\,(G_1) \cdot L\,(G_2)$. Draw a syntax tree of string $a\,b\,c\,d\,e\,f$, according to grammar G_3. Say if grammar G_3 is ambiguous or not, and explain why.

(c) (Optional) If the grammar G_3 previously defined is ambiguous, then define a new grammar G_4 that is not ambiguous, and shortly explain why it is not ambiguous.

---

2. One wishes one modeled the syntax of the declaration section of the variables in the C language, with some restrictions. Here are the relevant characteristics to be modeled:

   - a variable name is an identifier, symbolized by the terminal id
   - an integer constant is symbolized by the terminal num
   - the scalar type are int, char and float, the first one with the optional attribute unsigned
   - the structured types are the following ones:
     - the record or structure struct
     - the array of one or two dimensions, with elements of type scalar or record
   - it is also admitted to declare pointers, of any level
   - the sign "$*$" for declaring pointers is right-associative
   - the sign for declaring arrays (i.e., the square brackets "[ ]") is left-associative

   For anything that is not explicitly specified by the above list, do as it is specified by the standard of the C language.

   Here is an example of a few variable declarations:

```
char alpha, beta2, BIG;

unsigned int a, b, cc;

struct {
  float * f1;                  -- f1 is a pointer
  char * * omega;              -- omega is a pointer (of level 2)
} rec;

int matrix [10] [20];
```

   Answer the following questions:

   (a) Write a grammar, in the extended form (*EBNF*) and not ambiguous, that reasonably models the language fragment described above.

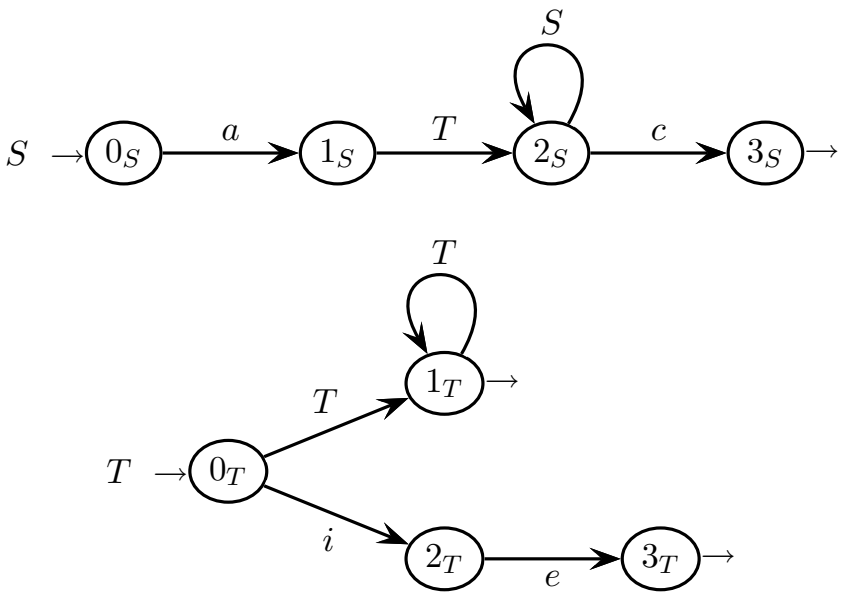
   (b) (Optional) Suppose one wanted to add the possibility of defining new types, by means of the typedef construct. Here is an example of a few type declarations:

```
typedef unsigned int UINT;     -- UINT is a new type
typedef char VECT [17];        -- VECT is a new type
typedef struct {
  ...
  ...
} * PREC, SUCC;                -- PREC e SUCC are new types
```

   Extend the grammar found at the point (a) (write an *EBNF* not ambiguous grammar). Show only the necessary new rules and / or those that are modified.

3 Syntax Analysis and Parsing 20%

1. Consider the grammar G defined by the recursive machines below (axiom S):



Answer the following questions:

(a) Build the smallest part of the *ELR* pilot graph of grammar G that allows to show that grammar G is not $ELL(1)$. Make sure that the m-states of the drawn pilot part contain all the necessary details, including the look-ahead sets.

(b) On the machines that represent grammar G, write the guide sets that prove that grammar G is not $ELL(1)$.

(c) Modify grammar G and obtain a variant grammar G' that is $ELL(1)$, of course without changing the generated language. On the machines of the new grammar G', show that the guide sets satisfy the $ELL(1)$ condition.

(d) (Optional) Continue to build the ELR pilot automaton of grammar G (started at point (a)), as much as it suffices to show that grammar G is not $ELR(1)$.

---

4 Translation and Semantic Analysis 20%

1. Consider the arithmetic expressions with the addition operator "+", but without parentheses, and with variables and constants symbolized by "v". Such expressions must be translated into a target language that, besides the standard addition operator "+", also has a fast addition function (called *fadd*), denoted as "$fadd(\ arg_1,\ arg_2\ )$", with two arguments. The translation tries to substitute the function *fadd* to the standard addition operator "+", which is less efficient. But the translation can be done in different ways, more or less optimized, as it is explained below.

   Answer the following questions:

   (a) One wishes one translated an addition expression by using the function *fadd* as much as possible, yet without using recursive function calls (nested functions) since the target language does not support them. Therefore the summands are pairwise translated with *fadd* starting from the left, and the last summand may be left alone with the operator "+". Here are a few examples:

source	*translation*
v	v
$v\ +\ v$	$fadd(\ v,\ v\ )$
$v\ +\ v\ +\ v$	$fadd(\ v,\ v\ )\ +\ v$
$v\ +\ v\ +\ v\ +\ v$	$fadd(\ v,\ v\ )\ +\ fadd(\ v,\ v\ )$

   Say if this translation is regular or syntactic, and consequently write the regular translation expression or the syntactic translation scheme that defines it.

   (b) One wishes one translated an addition expression by using only the function *fadd*, and by resorting to recursive function calls (nested functions) since the target language supports them. Addition must be considered ***right-associative***. Here are a few examples:

source	*translation*
v	v
$v\ +\ v$	$fadd(\ v,\ v\ )$
$v\ +\ v\ +\ v$	$fadd\big(\ v,\ fadd(\ v,\ v\ )\big)$

   Say if this translation is regular or syntactic, and consequently write the regular translation expression or the syntactic translation scheme that defines it.

   (c) (Optional) Discuss if the translations considered at points (a) and (b), are deterministic or not. Examine the options available to design deterministic automata (finite state or pushdown depending on the case) that compute such translations.

---

2. Consider the following syntactic support (axiom S):

$$\begin{cases} S \rightarrow E & E \rightarrow \text{`(' } E \text{ `)'} \\ E \rightarrow E \text{ `+' } E & E \rightarrow n \end{cases}$$

This syntax generates strings that represent simple additive expressions with parentheses, where the terminal n symbolizes the elementary summands.

Answer the following questions:

(a) Write an attribute grammar that computes, in a suitable attribute of the root, the maximum number of summands (a summand may be an elementary one or a subexpression between parentheses) that are present at the outer level or that are contained in a subexpression at an inner level. For instance, for the following expression:

$$\underbrace{n \; + \; \underbrace{(\, n \; + \; n \; + \; n \,)}_{3}}_{2}$$

the maximum is 3; for $n + (\, n + n + n \,) + n + n$ it is 4; and for $(\, n + n \,) + \big(\, n + (\, n + n + n + n + n \,) + n + n \,\big) + n + n$ it is 5. A solution can be written with only two attributes. See the tables on the next pages.

(b) Say if the attribute grammar defined at the previous point is one-sweep or not, say if it is L or not, and explain why.

(c) (Optional) Write a new attribute grammar variant of the previous one (but without changing the syntax support) that, in addition to the maximum number of summands as specified above, also computes the parenthetic nesting level where it is located the subexpression with the maximum number of summands. Review the sample cases listed before. For the following expression:

$$\underbrace{n \; + \; \underbrace{(\, n \; + \; n \; + \; n \,)}_{\text{num } 3 \quad \text{liv } 1}}_{\text{num } 2 \quad \text{liv } 0}$$

the maximum is 3 and the level is 1; for $n + (\, n + n + n \,) + n + n$ the maximum is 4 and the level is 0; and for $(\, n + n \,) + \big(\, n + (\, n + n + n + n + n \,) + n + n \,\big) + n + n$ the maximum is 5 and the level is 2.

If there are two or more (sub)expressions that have the maximum number of summands, but that are located at different nesting levels, the nesting level to be computed can be selected at choice among the levels of any such (sub)expressions. For instance, for the expression $n + (n + n + n) + n$ the maximum is 3, and the level may be 0 or 1.

A solution can be obtained by adding some attributes and the related semantic functions to the solution of point (a). See the tables in the next pages.

Say if the variant attribute grammar is one-sweep or not, say if it is L or not, and explain why.

attributes to be used for the grammar - question (a) - already defined

type	*name*	*domain*	*nonterm.*	*meaning*
left	a	integer	E	number of summands at the same nesting level in a (sub)expression
left	$amax$	integer	E, S	maximum number of summands at the same nesting level in a (sub)expression

attributes to be added for question (c)

type	*name*	*domain*	*nonterm.*	*meaning*

#	*syntax*	*semantics* - question (a)
1:	$S_0 \rightarrow E_1$	
2:	$E_0 \rightarrow E_1$ ‘+’ E_2	
3:	$E_0 \rightarrow$ ‘(’ E_1 ‘)’	
4:	$E_0 \rightarrow n$	

#	*syntax*	*semantics* - question (c)
1:	$S_0 \rightarrow E_1$	
2:	$E_0 \rightarrow E_1$ ‘+’ E_2	
3:	$E_0 \rightarrow$ ‘(’ E_1 ‘)’	
4:	$E_0 \rightarrow n$	